

063-0605-23L

Computational Structural Design I

Computational 2D Graphic Statics

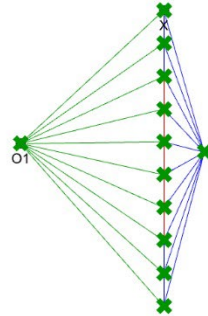
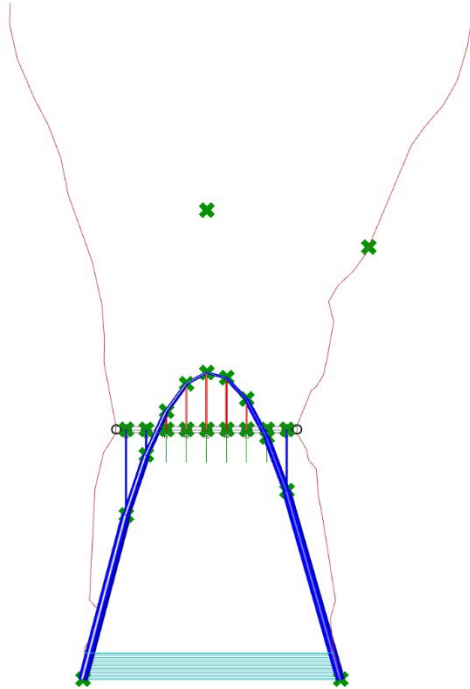
Dr. Lluís Enrique Monzó

Block Research Group (Prof. Dr. Philippe Block)

HS2023

Week 10 Friday, November 24th

- | | |
|---------------|--|
| 9:45 – 10:30 | Recap of last module's exercise
Lecture: Algebraic Graphic statics |
| 10:30 – 10:45 | Break |
| 10:45 – 11:30 | Tutorial: Interactive Graphic Statics |
| 11:30 – 11:45 | Break |
| 11:45 – 12:30 | Tutorial: Interactive Graphic Statics |



```
import rhinoscriptsyntax as rs
```

```
#1.a INPUT LOADS
```

```
#Task 1
```

```
#input
```

```
Lp_anc=[p1,p2,p3,p4,p5,p6,p7,p8,p9]
```

```
Ln_mag=[n1,n2,n3,n4,n5,n6,n7,n8,n9]
```

```
#vector loads
```

```
Lv_load=[]
```

```
for i in range (0,len(Lp_anc)):
```

```
.....v=[0,Ln_mag[i],0]
```

```
.....Lv_load.append(v)
```

```
#LOA (Line Of Action)
```

```
Ll_LOA=[]
```

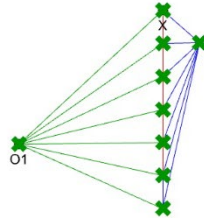
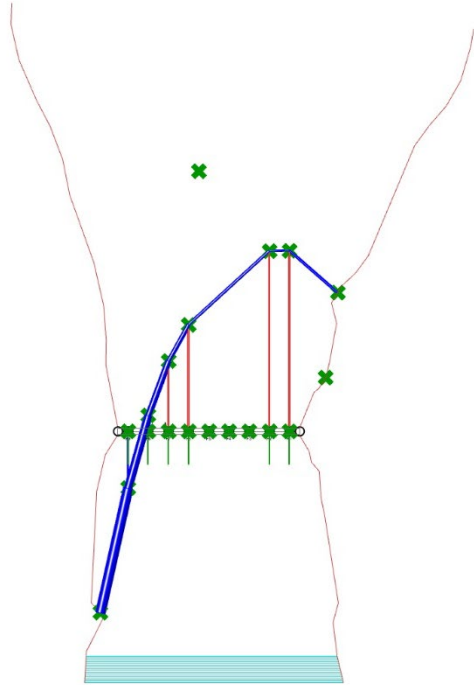
```
for i in range (0,len(Lp_anc)):
```

```
.....p=rs.CopyObject(Lp_anc[i])
```

```
.....rs.MoveObject(p,[0,-10,0])
```

```
.....LOA=rs.AddLine(Lp_anc[i],p)
```

```
.....Ll_LOA.append(LOA)
```



```
import rhinoscriptsyntax as rs
```

```
#1.a INPUT LOADS
```

```
#input
```

```
Lp_anc_in=[p1,p2,p3,p4,p5,p6,p7,p8,p9]
```

```
Ln_mag_in=[n1,n2,n3,n4,n5,n6,n7,n8,n9]
```

```
#Task 2
```

```
Lp_anc=[]
```

```
Ln_mag=[]
```

```
for i in range (0,len(Lp_anc_in)):
```

```
    ...if Ln_mag_in[i] != 0:
```

```
        ...Lp_anc.append(Lp_anc_in[i])
```

```
        ...Ln_mag.append(Ln_mag_in[i])
```

```
#vector loads
```

```
Lv_load=[]
```

```
for i in range (0,len(Lp_anc)):
```

```
    ...v=[0,Ln_mag[i],0]
```

```
    ...Lv_load.append(v)
```

```
#LOA (Line Of Action)
```

```
L1_LOA=[]
```

```
for i in range (0,len(Lp_anc)):
```

```
    ...p=rs.CopyObject(Lp_anc[i])
```

```
    ...rs.MoveObject(p,[0,-10,0])
```

```
    ...LOA=rs.AddLine(Lp_anc[i],p)
```

```
    ...L1_LOA.append(LOA)
```

```
import rhinoscriptsyntax as rs
```

```
#1.a INPUT LOADS
```

```
n=[p1,p2,p3,p4,p5,p6,p7,p8,p9]
n=[n1,n2,n3,n4,n5,n6,n7,n8,n9]
```

```
    and 3
    ]
    ]

    range (0,len(Lp_anc_in)):
    n_mag_in[i] != 0:
    if M == i:
        Lp_anc.append(Lp_anc_in[i])
        Ln_mag.append(Ln_mag_in[i]+Q)
    else:
        Lp_anc.append(Lp_anc_in[i])
        Ln_mag.append(Ln_mag_in[i])
```

```
loads
[[]]
    range (0,len(Lp_anc)):
    Ln_mag[i],0]
    load.append(v)
```

```
ne Of Action)
```

```
    range (0,len(Lp_anc)):
    .CopyObject(Lp_anc[i])
    rs.MoveObject(p,[0,-10,0])
    LOA=rs.AddLine(Lp_anc[i],p)
    Ll_LOA.append(LOA)
```

```

import rhinoscriptsyntax as rs

#5&6. SENSE AND VISUALIZATION

#check angle
Langle=[]
for i in range (0,len(Ll_form)):
    ....angle=rs.Angle2(Ll_form[i],Ll_force[i])
    ....Langle.append(round(angle[0]))

#colors in force diagram and pipes in form diagram
x=0.01
Ll_force_ten=[]
Ll_force_com=[]
Lpi_ten=[]
Lpi_com=[]
for i in range (0,len(Ll_form)):
    ....pi=rs.AddPipe(Ll_form[i],0,x*rs.CurveLength(Ll_force[i]),0,1)
    ....if Langle[i] ==0:
    ....Ll_force_ten.append(Ll_force[i])
    ....Lpi_ten.append(pi[0])
    ....else:
    ....Ll_force_com.append(Ll_force[i])
    ....Lpi_com.append(pi[0])

```

```

import rhinoscriptsyntax as rs
import math as m

```

```
#5&6. SENSE AND VISUALIZATION
```

```

#check angle
Langle=[]
for i in range (0,len(Ll_form)):
    ....angle=rs.Angle2(Ll_form[i],Ll_force[i])
    ....Langle.append(round(angle[0]))

```

```
#Task 4
```

```
#colors in force diagram and pipes in form diagram
```

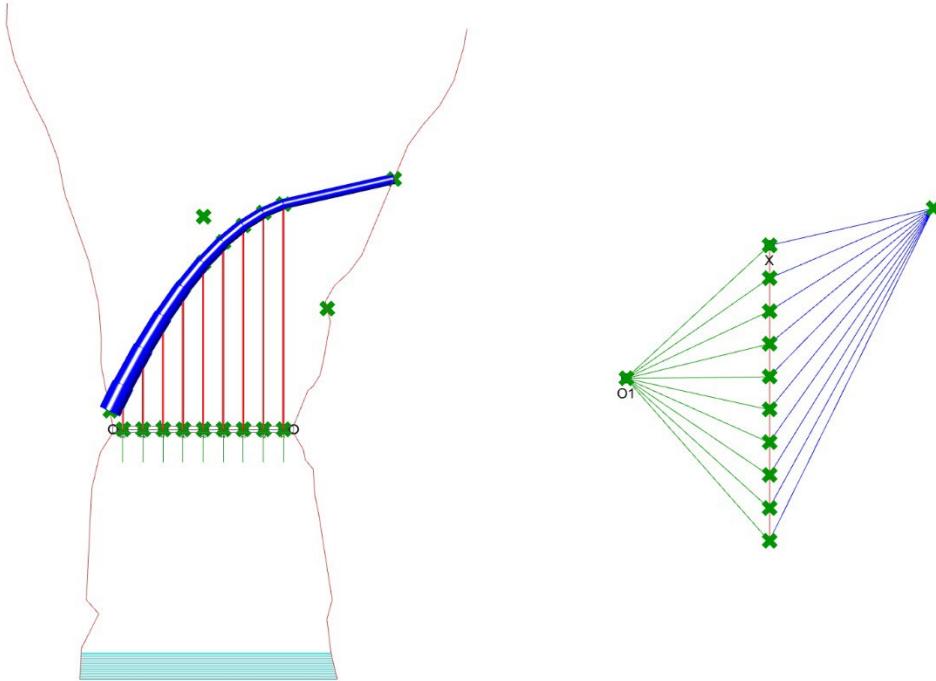
```

x=1
Ll_force_ten=[]
Ll_force_com=[]
Lpi_ten=[]
Lpi_com=[]

Scd=235/1.05
Std=1670/1.05

for i in range (0,len(Ll_form)):
    ....if Langle[i] ==0:
    ....Areq=(rs.CurveLength(Ll_force[i])*1000/Std)/1000000
    ....Rad=m.sqrt(Areq/3.14)
    ....pi=rs.AddPipe(Ll_form[i],0,x*Rad,0,1)
    ....Ll_force_ten.append(Ll_force[i])
    ....Lpi_ten.append(pi[0])
    ....else:
    ....Areq=(rs.CurveLength(Ll_force[i])*1000/Scd)/1000000
    ....Rad=m.sqrt(Areq/3.14)
    ....pi=rs.AddPipe(Ll_form[i],0,x*Rad,0,1)
    ....Ll_force_com.append(Ll_force[i])
    ....Lpi_com.append(pi[0])

```



```
import rhinoscriptsyntax as rs

#2. SUPPORTS AND REACTIONS

#creating lines of reaction forces in form diagram
l1=rs.AddLine(p_R_m,sp_left)
l2=rs.AddLine(p_R_m,sp_right)

#building reaction forces in force diagram
v1=rs.VectorCreate(Lp_loadline[-1],p_R_m)
l1_m=rs.MoveObject(l1,v1)

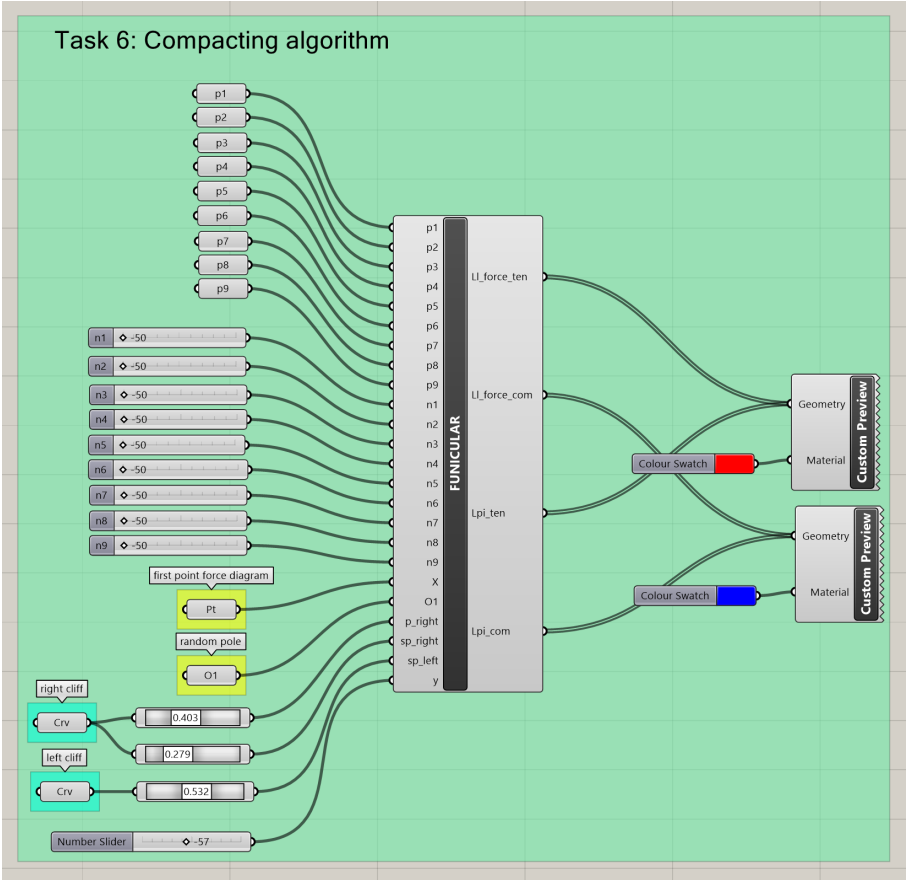
v2=rs.VectorCreate(Lp_loadline[0],p_R_m)
l2_m=rs.MoveObject(l2,v2)

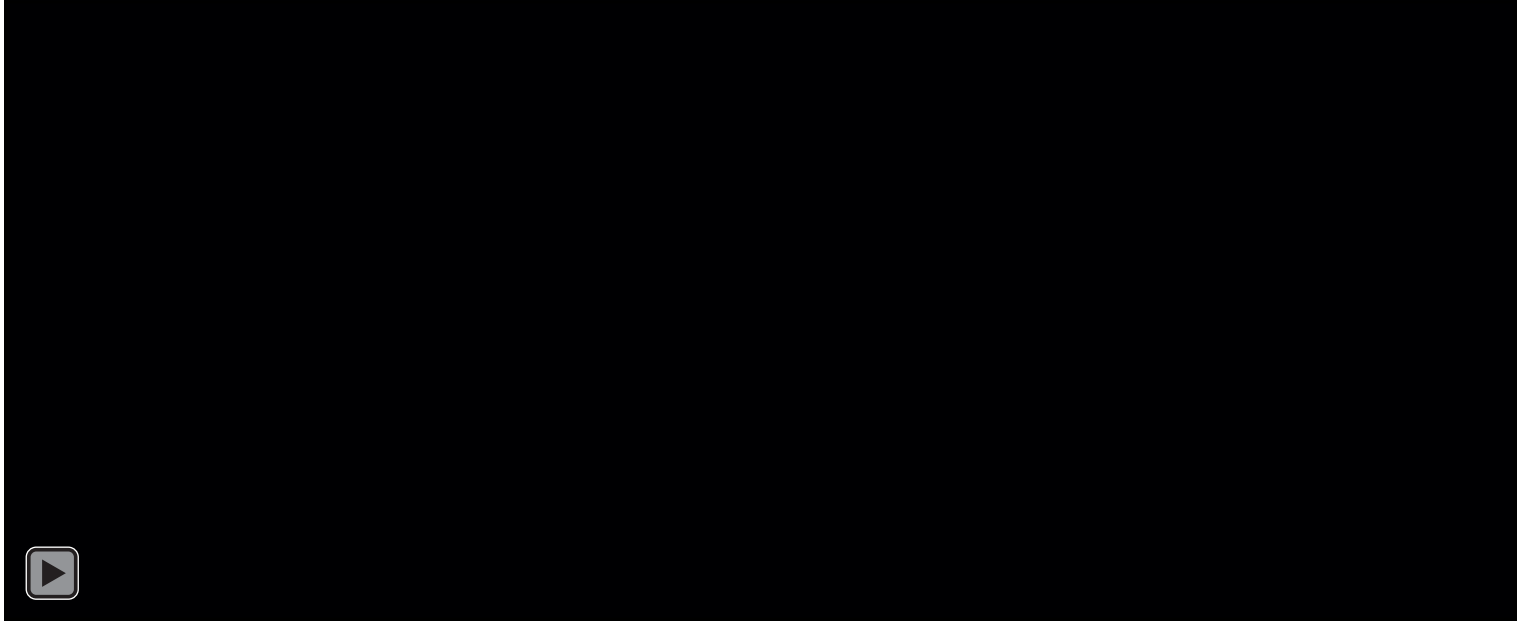
O2=rs.LineLineIntersection(l1_m,l2_m)[0]

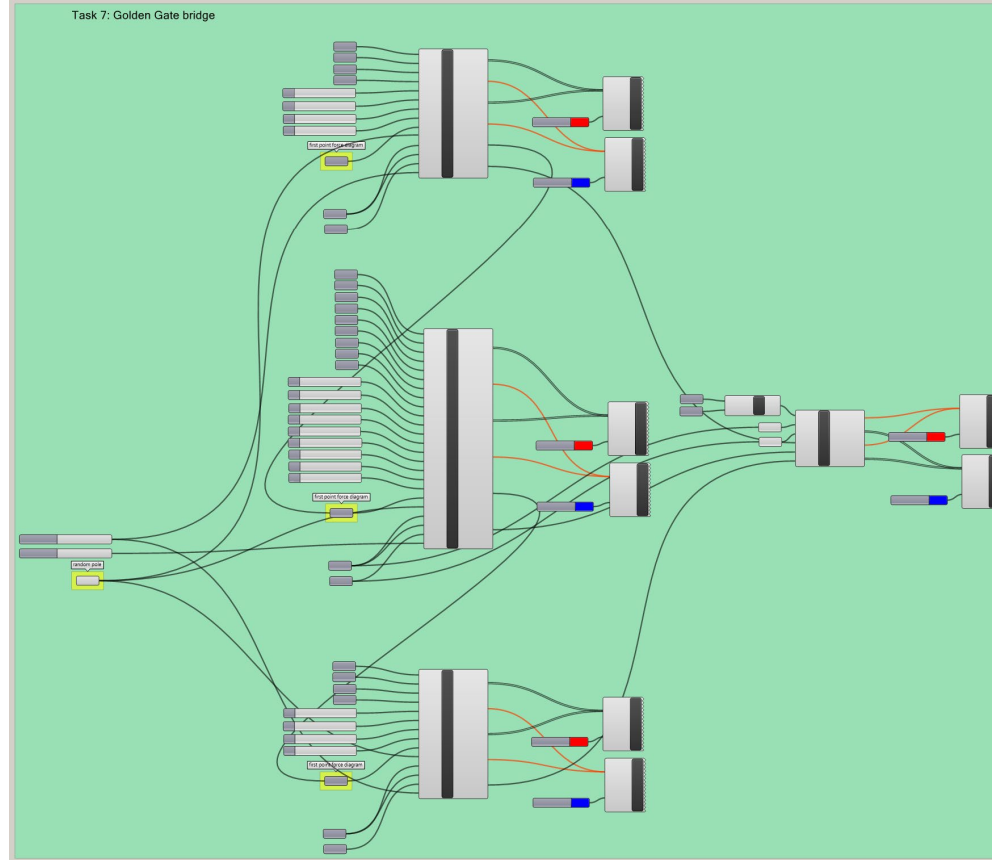
#Task 5
l_CS=rs.AddLine(sp_left,sp_right)
v=rs.VectorCreate(O2,sp_left)
l_CS_copy=rs.CopyObject(l_CS,v)

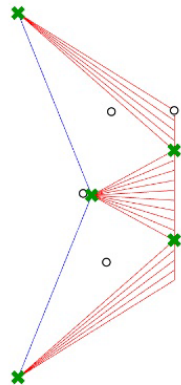
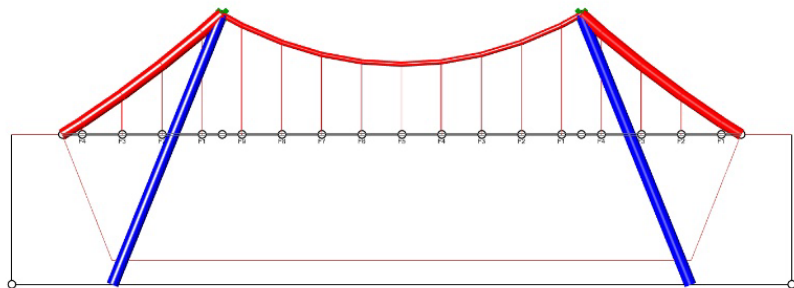
a1=rs.AddLine(Lp_loadline[0],Lp_loadline[-1])
a1_move=rs.MoveObject(a1,[50,0,0])

O2=rs.LineLineIntersection(l_CS_copy, a1_move)[0]
```









```
import rhinoscriptsyntax as rs
import math as m
```

```
#Task 7
```

```
l1_force=rs.AddLine(02_2,02_1)
l2_force=rs.AddLine(02_3,02_2)
```

```
v1=rs.VectorCreate(sp_right,02_1)
a11_form=rs.CopyObject(l1_force,v1)
ip1=rs.LineLineIntersection(l_terrain,a11_form)[0]
l1_form=rs.AddLine(sp_right,ip1)
```

```
v2=rs.VectorCreate(sp_left,02_2)
a12_form=rs.CopyObject(l2_force,v2)
ip2=rs.LineLineIntersection(l_terrain,a12_form)[0]
l2_form=rs.AddLine(sp_left,ip2)
```

```
L1_force=[l1_force,l2_force]
L1_form=[l1_form,l2_form]
```

```
#check angle
```

```
Langle=[]
for i in range(0,len(L1_form)):
    angle=rs.Angle2(L1_form[i],L1_force[i])
    Langle.append(round(angle[0]))
```

```
#Task 4
```

```
#colors in force diagram and pipes in form diagram
```

```
x=1
```

```
L1_force_ten=[]
```

```
L1_force_com=[]
```

```
Lp1_ten=[]
```

```
Lp1_com=[]
```

```
Scd=235/1.05
```

```
Std=1670/1.05
```

```
for i in range(0,len(L1_form)):
```

```
    if Langle[i] ==0:
        Areq=(rs.CurveLength(L1_force[i])*1000/Std)/1000000
```

```
        Rad=m.sqrt(Areq/3.14)
```

```
        pi=rs.AddPipe(L1_form[i],0,x*Rad,0,1)
```

```
        L1_force_ten.append(L1_force[i])
```

```
        Lp1_ten.append(pi[0])
```

```
    else:
```

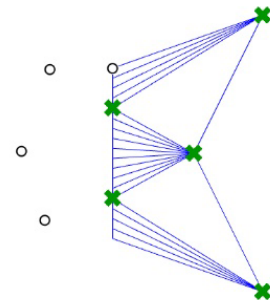
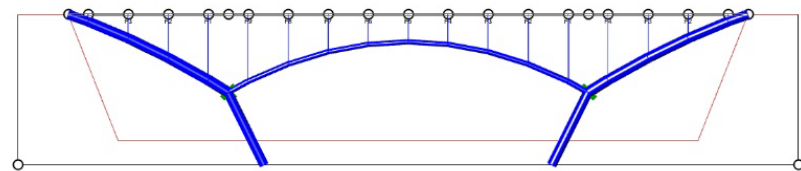
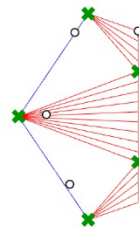
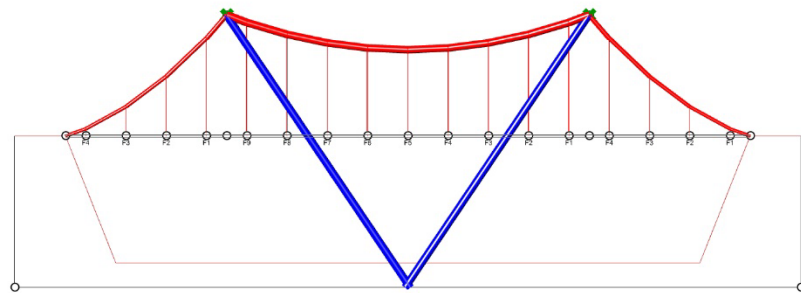
```
        Areq=(rs.CurveLength(L1_force[i])*1000/Scd)/1000000
```

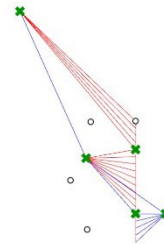
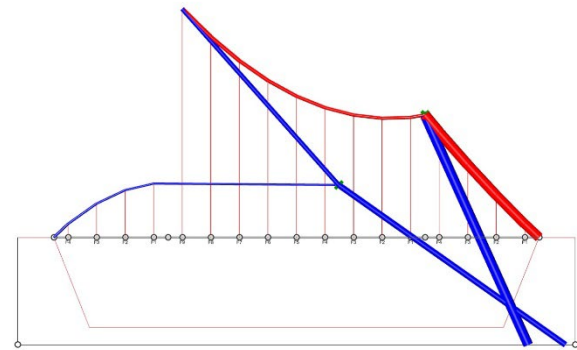
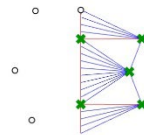
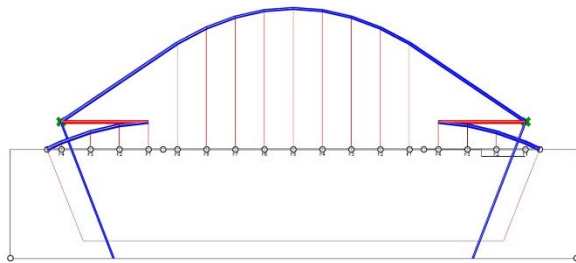
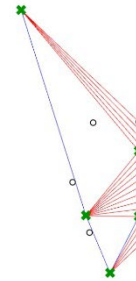
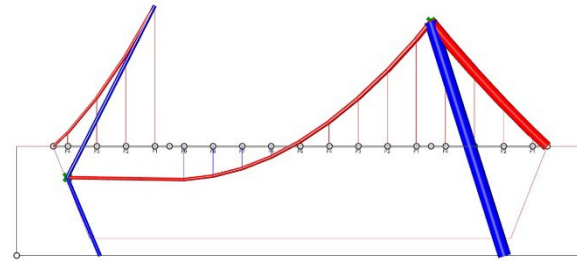
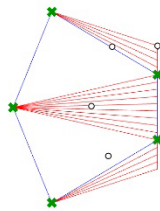
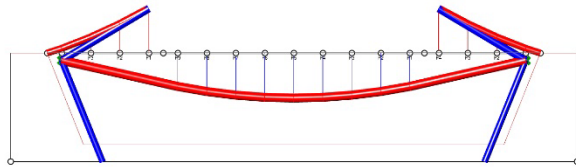
```
        Rad=m.sqrt(Areq/3.14)
```

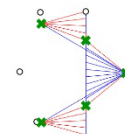
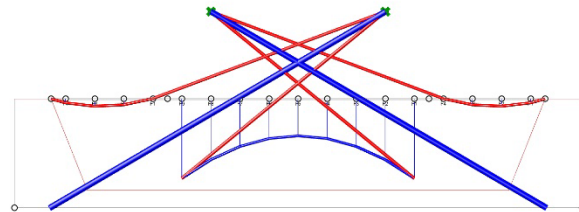
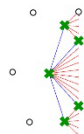
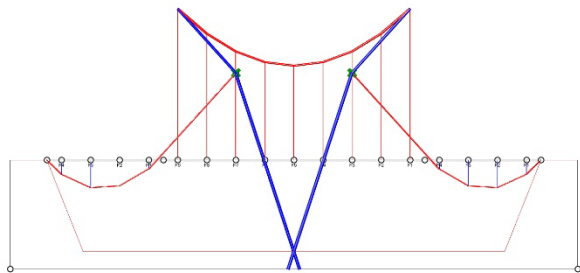
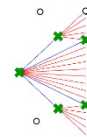
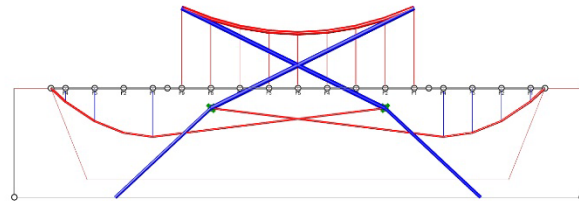
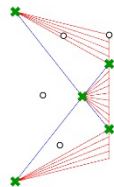
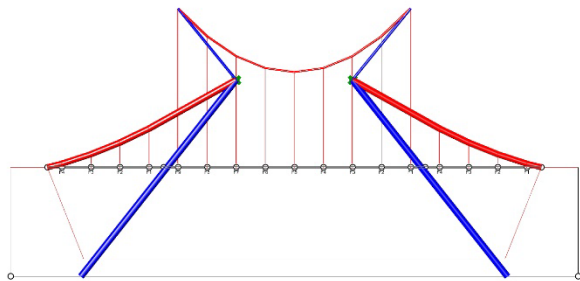
```
        pi=rs.AddPipe(L1_form[i],0,x*Rad,0,1)
```

```
        L1_force_com.append(L1_force[i])
```

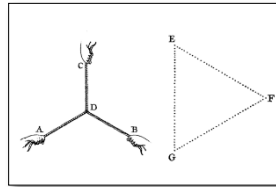
```
        Lp1_com.append(pi[0])
```



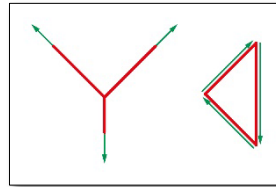




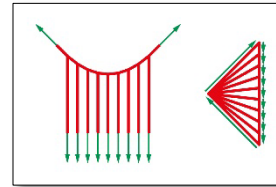
Module	Week		Topic
I	Week 1 (22/9)	Lecture	Introduction to graphic statics
		Tutorial	Fundamentals of graphic statics Introduction to Rhino and Grasshopper
		Intro exercise	EX I-1: Graphic statics in Rhino EX I-2: Grasshopper basics
II	Week 2 (29/9)	Submission	Submission of EX I-1 and EX I-2 and quick review of the solution
		Lecture	Algorithmic design & thinking
		Tutorial 1	Single node bridge in Grasshopper
	Week 3 (06/10)	Intro exercise	EX II-1 : Single node bridge
		Work session	
	Week 4 (13/10)	Submission	Submission of EX II-1 and quick review of the solution
		Tutorial 2	Form finding of funicular structures using graphic statics Form finding of funicular structures using graphic statics in Grasshopper
	Week 5 (20/10)	Intro exercise	EX II-2 : Form finding of funicular structures
		Work session	
Week 6: seminar week			
III	Week 7 (03/11)	Submission	Submission of EX II-2 and quick review of the solution
		Lecture	Computational graphic statics
		Tutorial 1	Programming in Grasshopper Python
		Intro exercise	EX III-1: Programming in Grasshopper Python
	Week 8 (10/11)	Tutorial 2	Form finding of funicular structures in Grasshopper Python
		Intro exercise	EX III-2: Form finding of funicular structures in Grasshopper Python
		Work session	EX III-1 and EX III-2
	Week 9 (17/11)	Work session	EX III-1 and EX III-2
IV	Week 10 (24/11)	Submission	Submission of EX III-1 and EX III-2 and quick review of the solution
		Lecture	Algebraic Graph Statics (AGS)
		Tutorial 1	Analysis of trusses using graphic statics Analysis of trusses using Interactive Graphic Statics (IGS2)
		Intro exercise	EX IV-1: Analysis of trusses using Interactive Graphic Statics (IGS2)
	Week 11 (01/12)	Tutorial 2	Form-finding using Interactive Graphic Statics (IGS2)
		Intro exercise	EX IV-2: Form-finding using Interactive Graphic Statics (IGS2)
		Work session	EX IV-1 and EX IV-2
	Week 12 (08/12)	Work session	EX IV-1 and EX IV-2
		Submission	Submission of EX IV-1 and EX IV-2



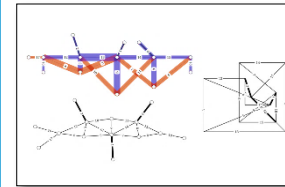
Module I
Introduction to Graphic statics



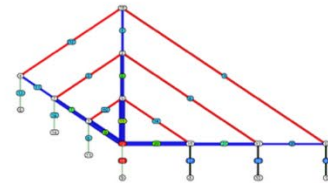
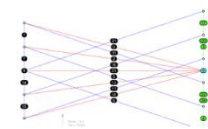
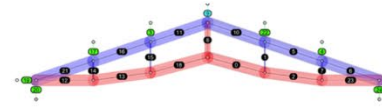
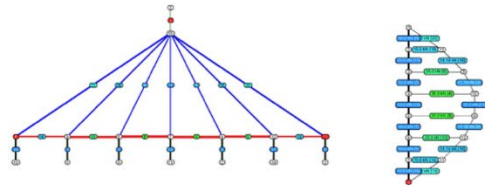
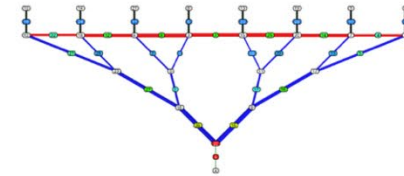
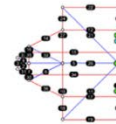
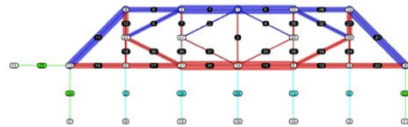
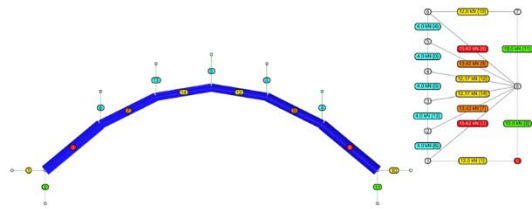
Module II
Procedural Graphic Statics



Module III
Programming Graphic Statics



Module IV
Algebraic Graph Statics

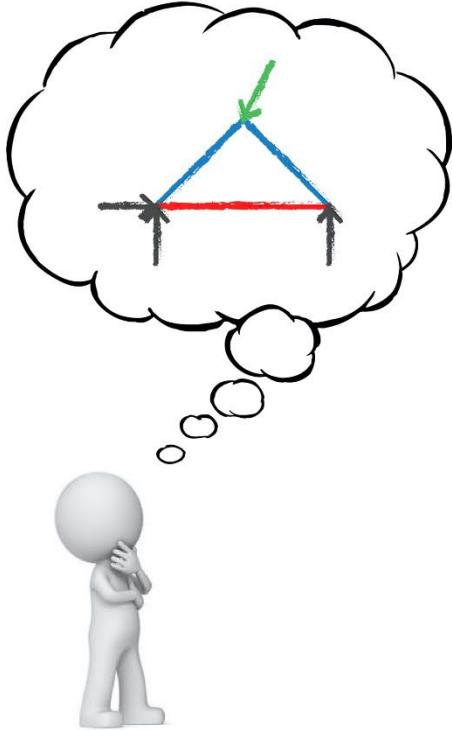


Algebraic Graph(ic) Statics

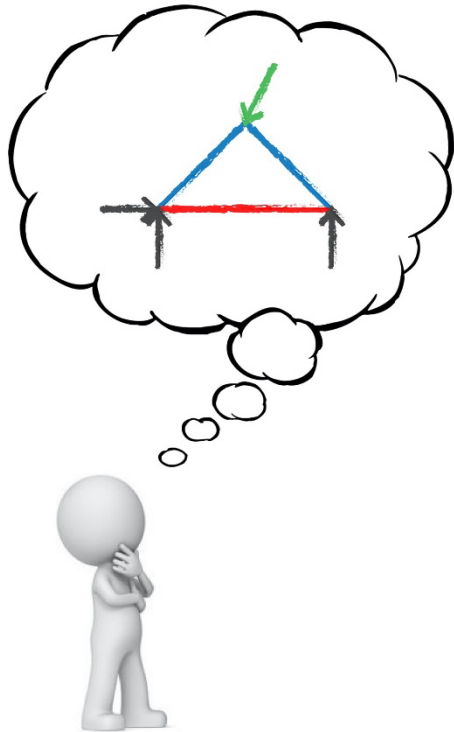
Friday, November 24th, 2023

Dr. Lluís Enrique Monzó

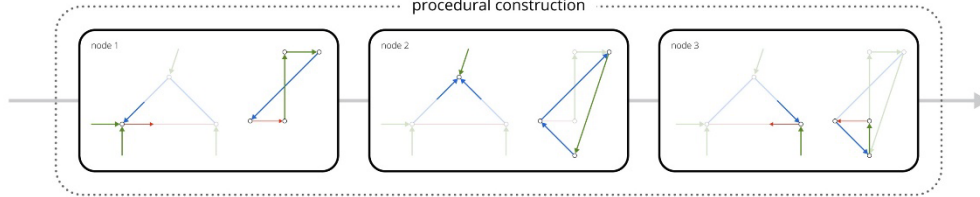
form diagram



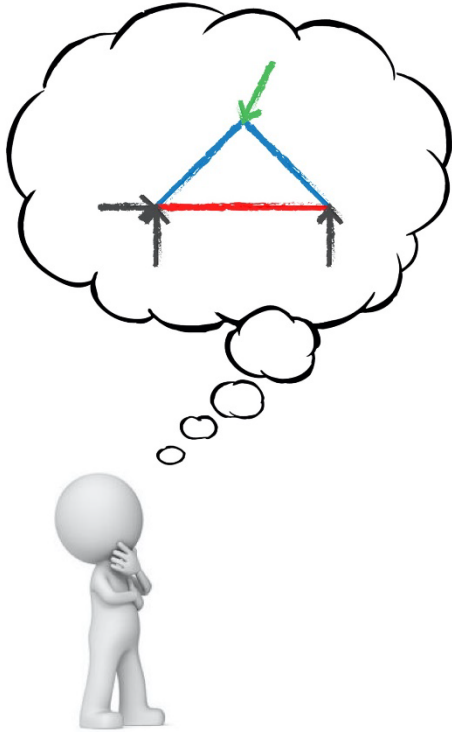
form diagram



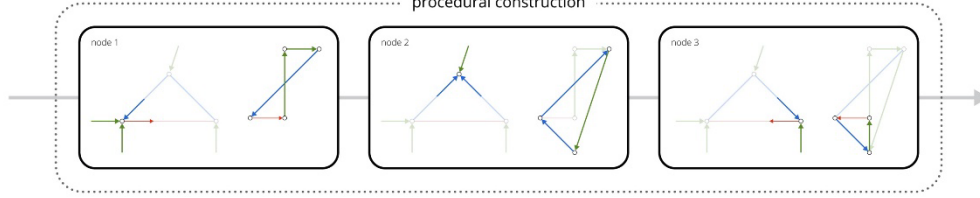
procedural construction



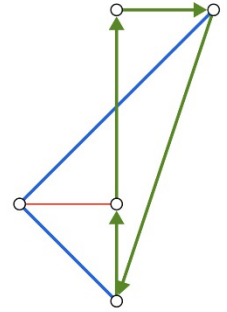
form diagram



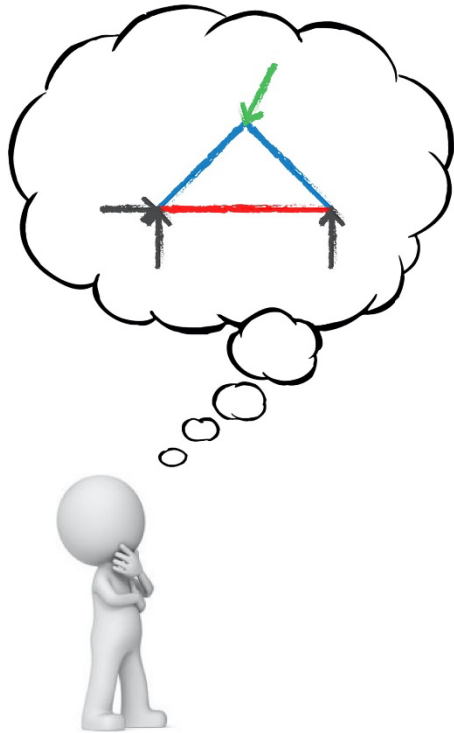
procedural construction



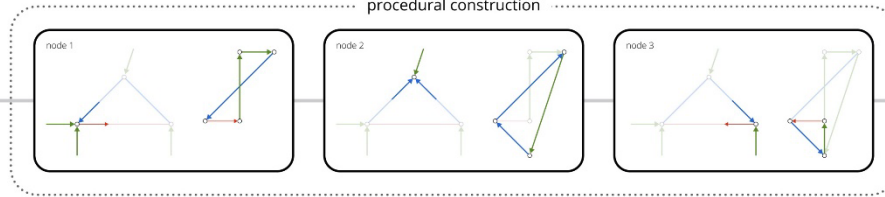
force diagram



form diagram

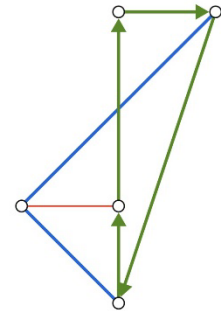


procedural construction

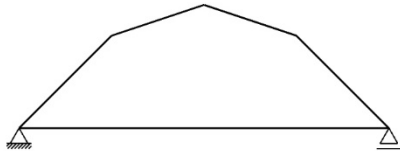
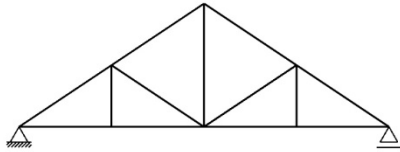


algebraic construction

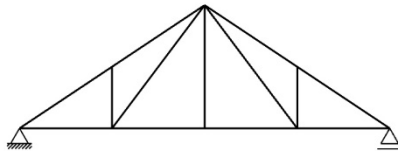
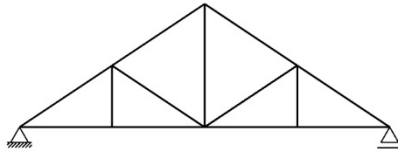
force diagram



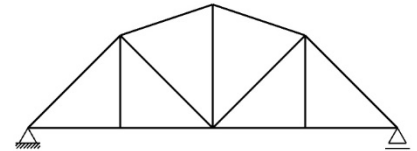
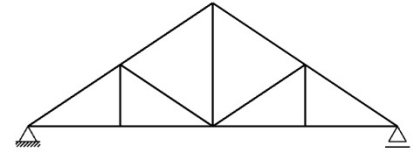
Typology



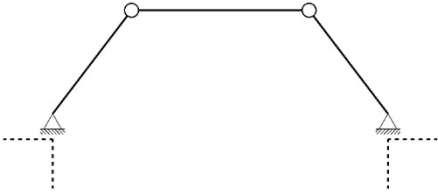
Topology



Geometry



Cables / Arches



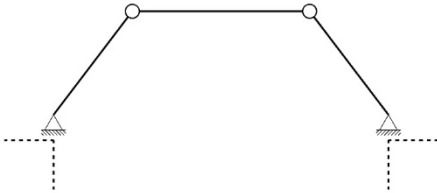
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Cables / Arches



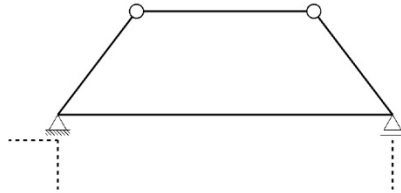
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



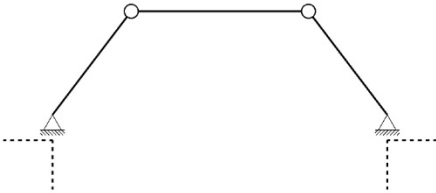
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Cables / Arches



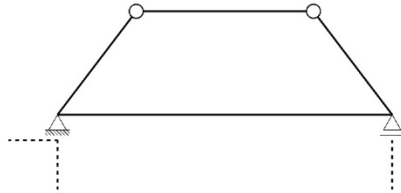
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



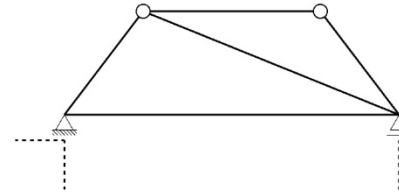
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

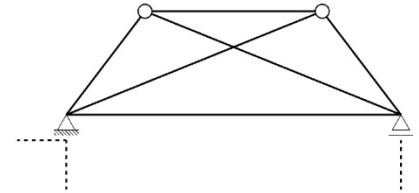


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

Stable!
 and
 statically determinate!



b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

$$6 + 3 > 2 \cdot 4$$

Stable!
 but
 statically indeterminate!

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

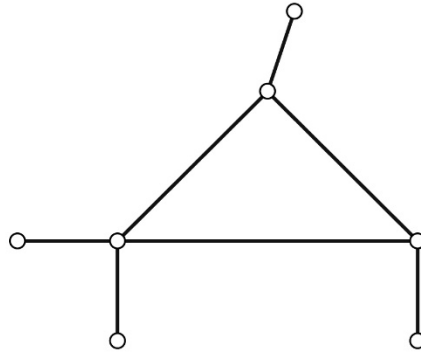
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

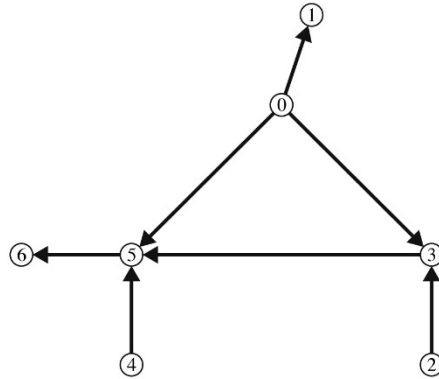
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

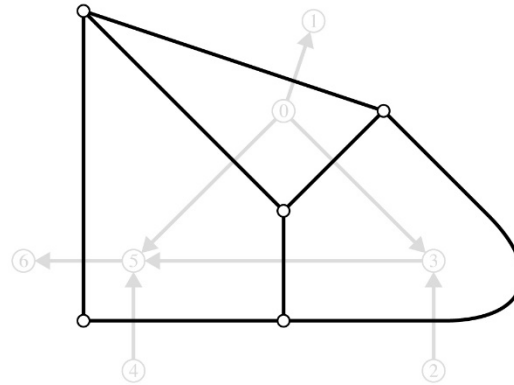
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

[Dual graph](#)

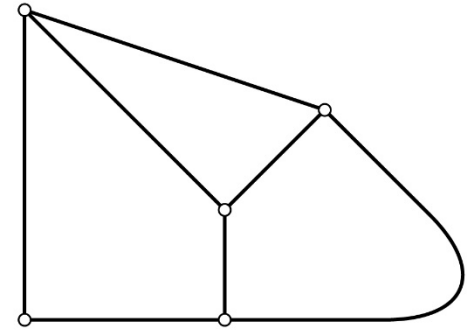
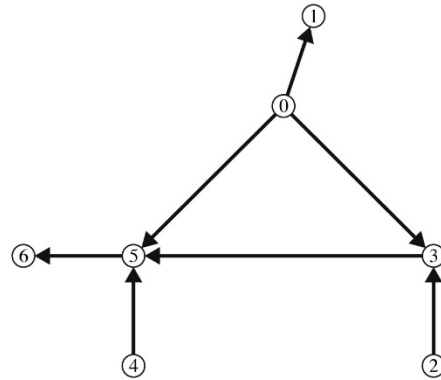
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

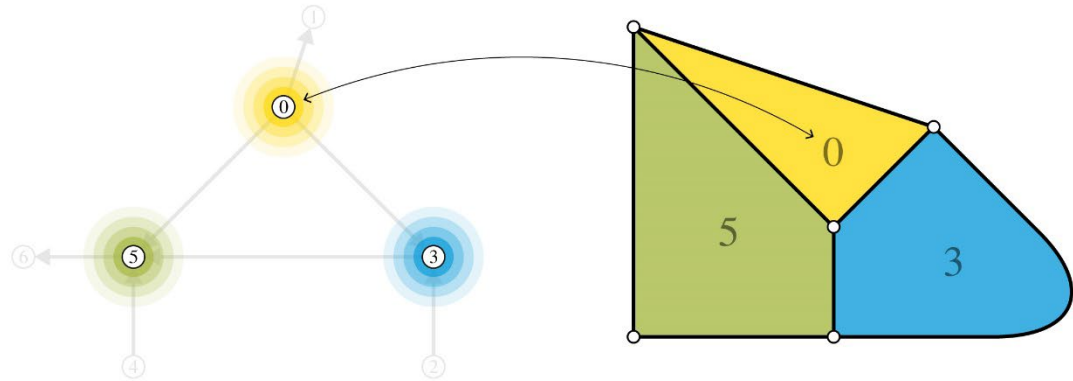
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

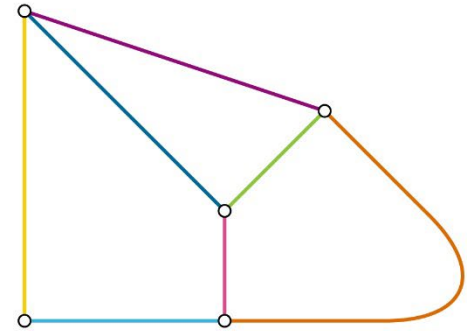
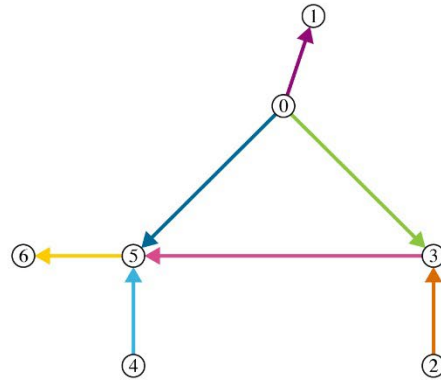
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

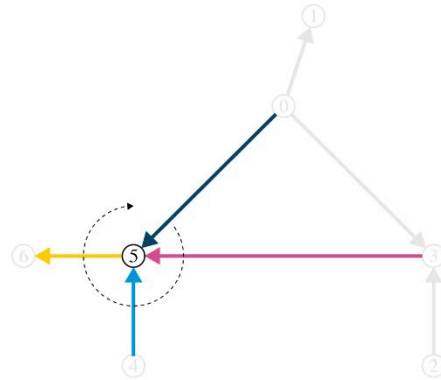
[Directed force graph](#)

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

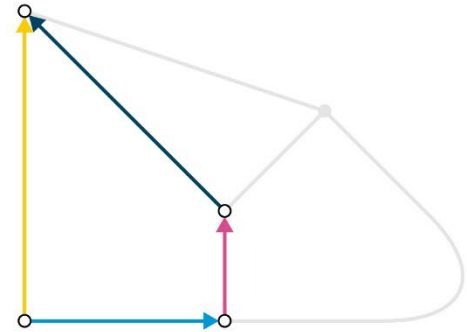
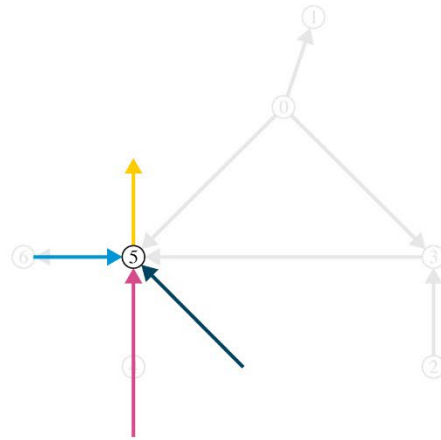
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

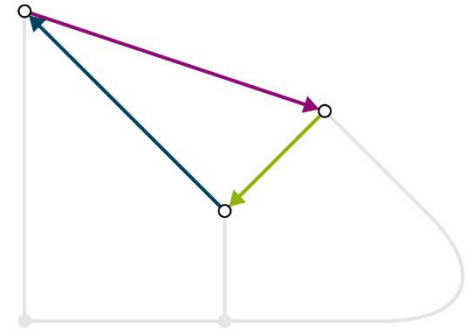
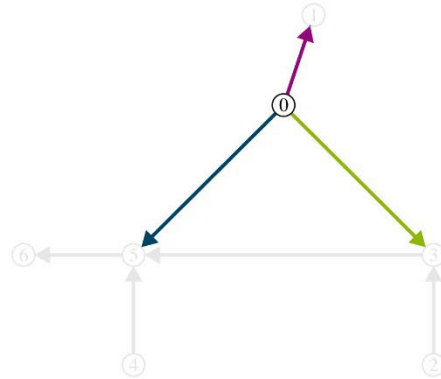
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

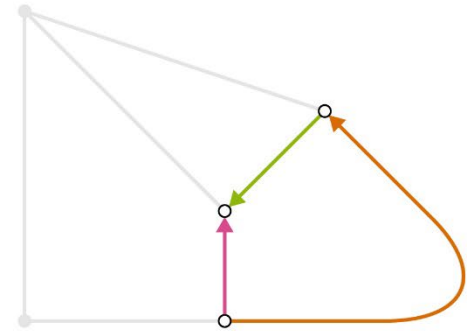
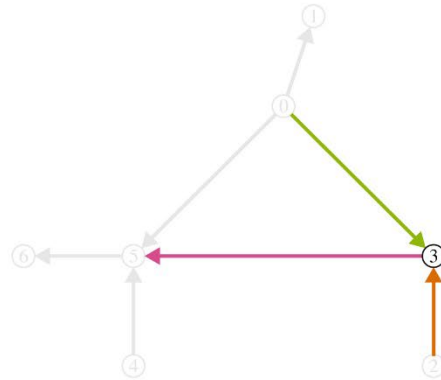
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

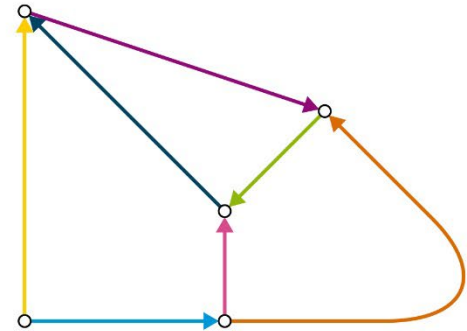
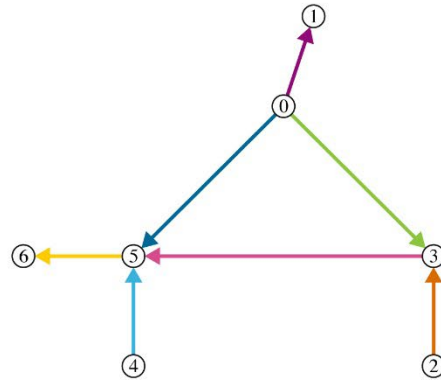
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

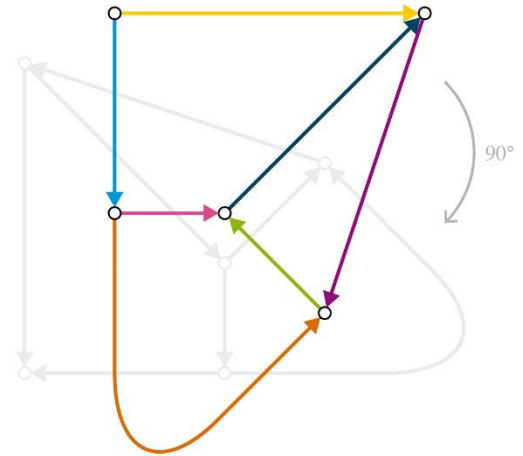
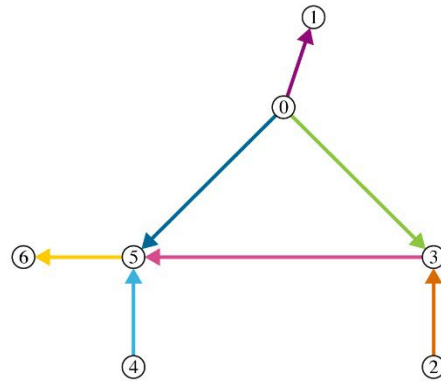
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

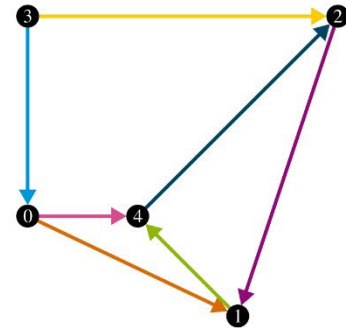
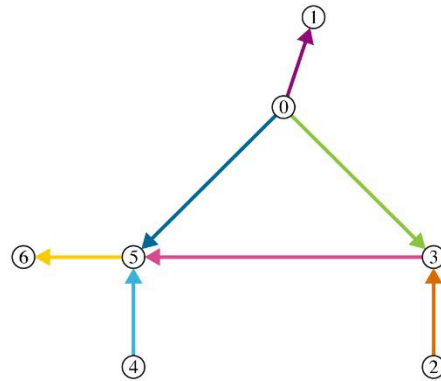
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

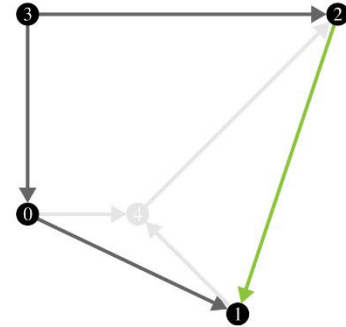
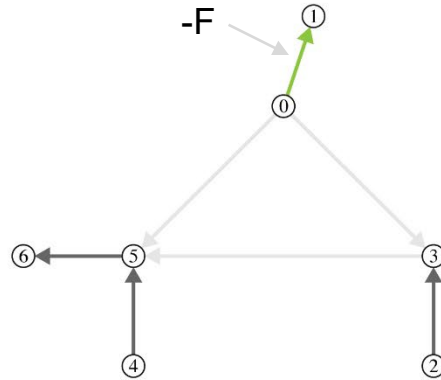
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

Directed force graph

3. Assign forces

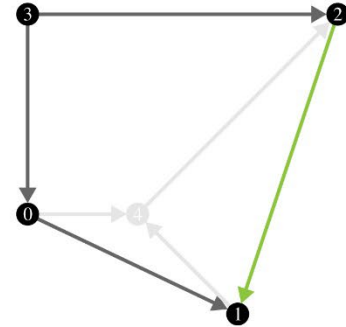
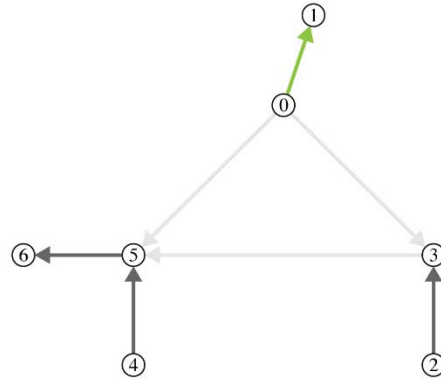
4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs

$$C_{ij} = \begin{cases} +1 & \text{if vertex } i \text{ is the head of edge } j \\ -1 & \text{if vertex } i \text{ is the tail of edge } j \\ 0 & \text{otherwise} \end{cases}$$

$$C_{ij}^* = \begin{cases} +1 & \text{if the face cycle traverses} \\ & \text{edge } j \text{ in the same direction} \\ & \text{as its orientation} \\ -1 & \text{if in the opposite direction} \\ 0 & \text{otherwise} \end{cases}$$



Closed polygon constraint

$$\begin{cases} C_i u^* = 0 \\ C_i v^* = 0 \end{cases}$$

Parallel constraint

$$\begin{cases} u^* = Qu \\ v^* = Qv \end{cases}$$

$$\begin{cases} C_i Uq = 0 \\ C_i Vq = 0 \end{cases}$$

equilibrium matrix

$$Aq = 0, \quad A = \begin{bmatrix} C_i U \\ C_i V \end{bmatrix}$$

1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

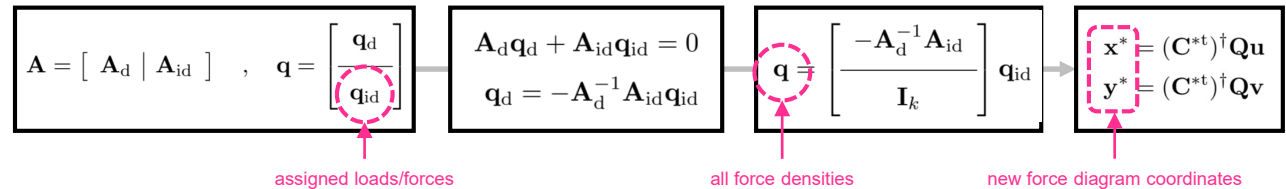
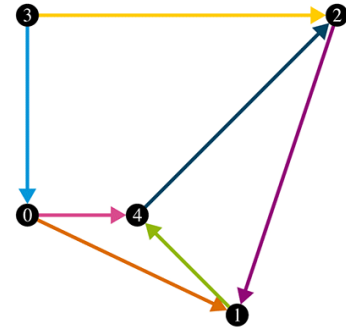
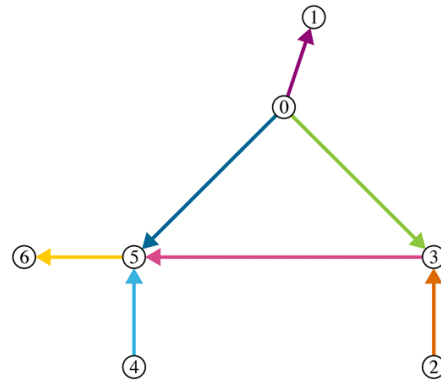
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

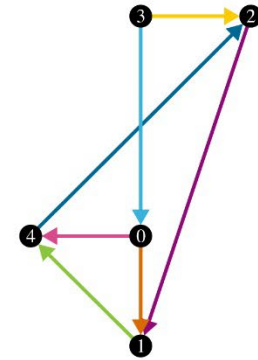
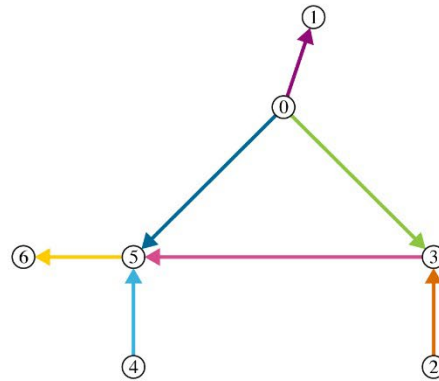
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

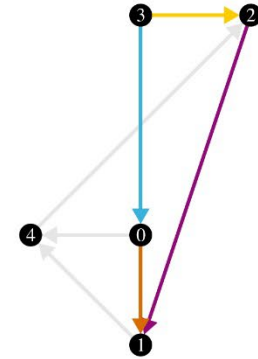
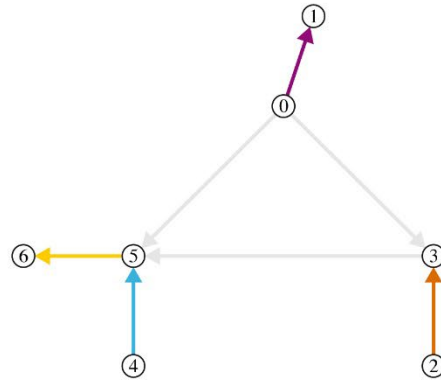
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

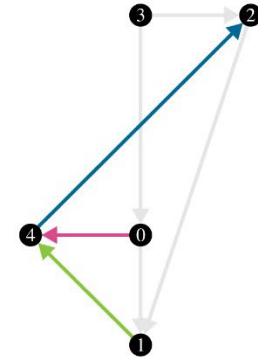
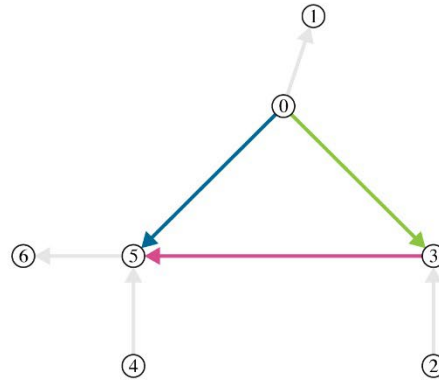
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

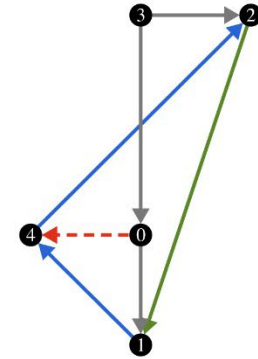
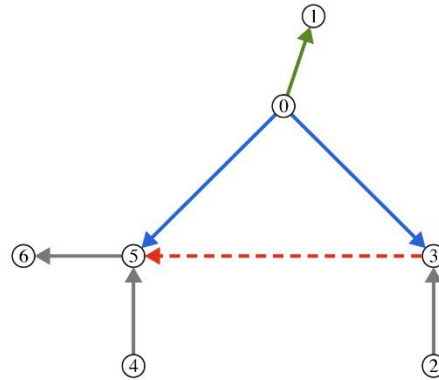
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

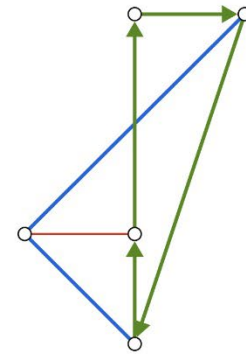
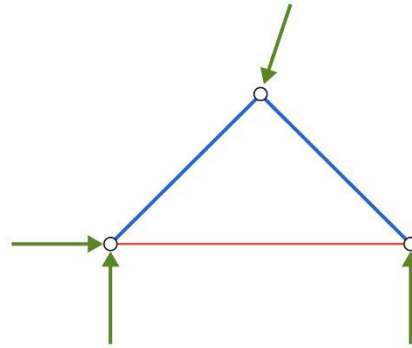
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

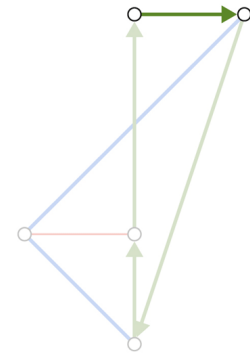
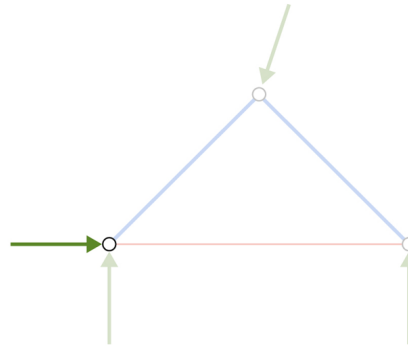
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

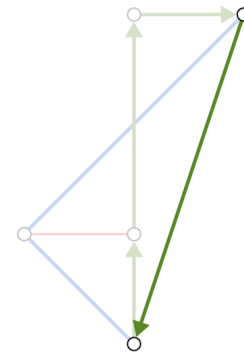
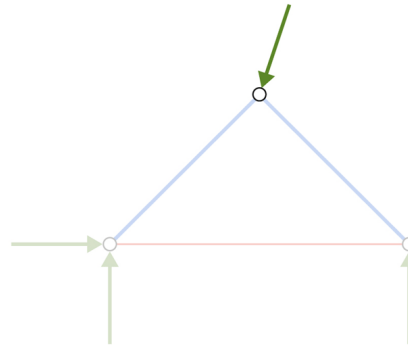
Directed force graph

3. Assign forces

4. Solve equilibrium matrix

5. Update force graph

6. Reciprocal form & force graphs



1. Form graph

Connectivity (edges & vertices)

Directed form graph

2. Force graph

Dual graph

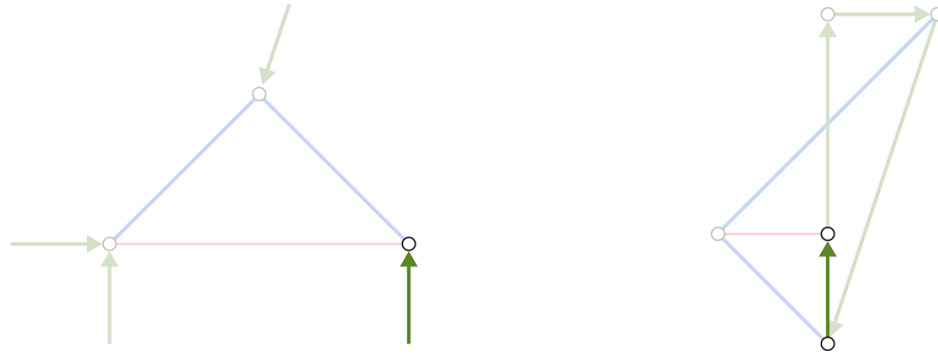
Directed force graph

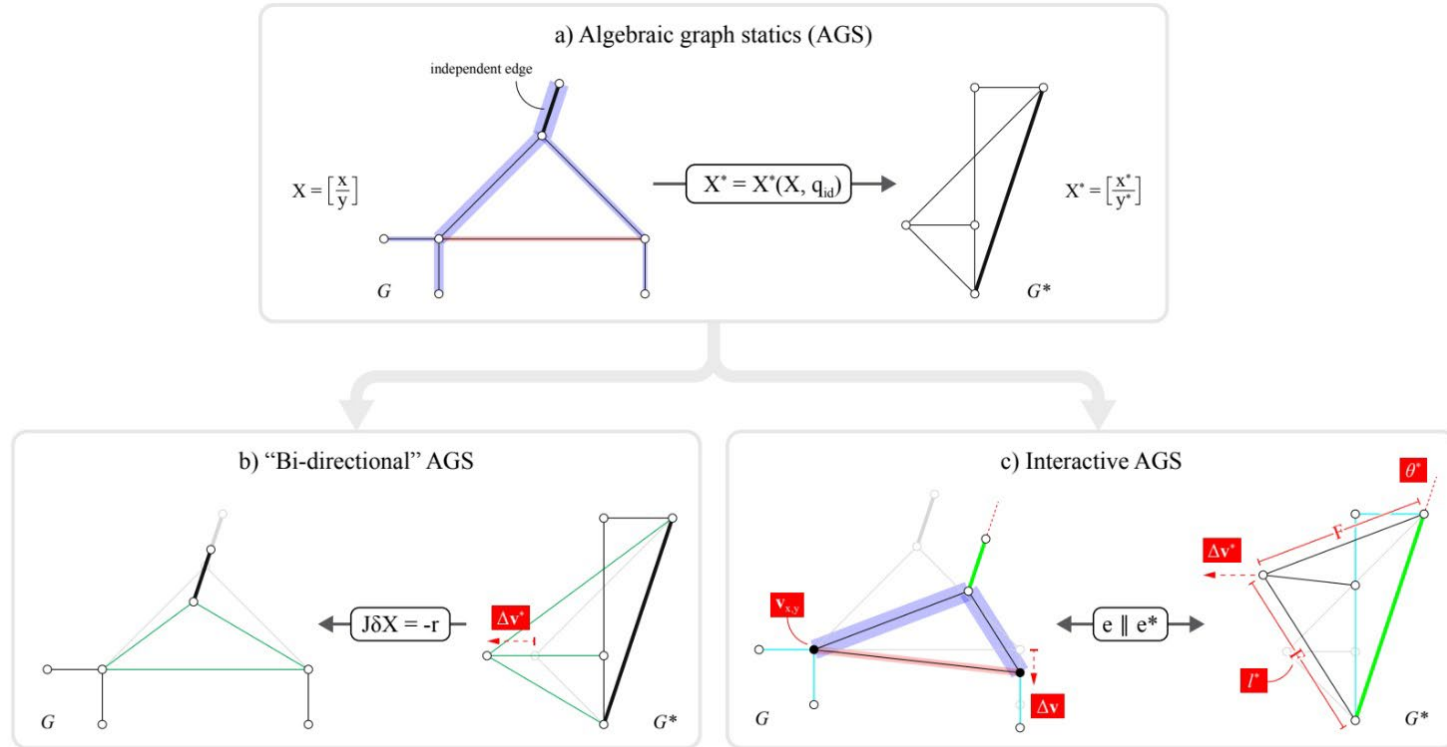
3. Assign forces

4. Solve equilibrium matrix

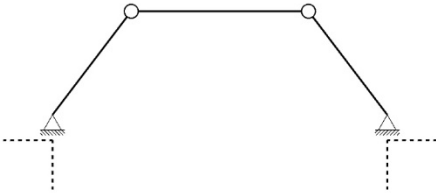
5. Update force graph

6. Reciprocal form & force graphs





Cables / Arches



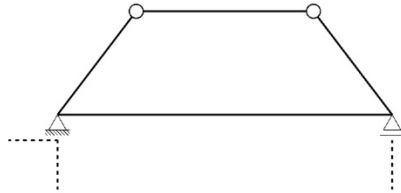
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



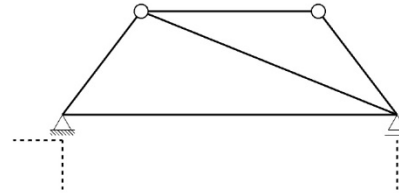
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

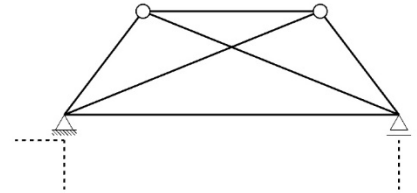


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

Stable!
 and
 statically determinate!



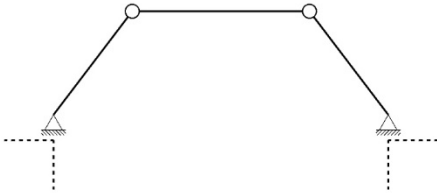
b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

$$6 + 3 > 2 \cdot 4$$

Stable!
 but
 statically indeterminate!

Cables / Arches



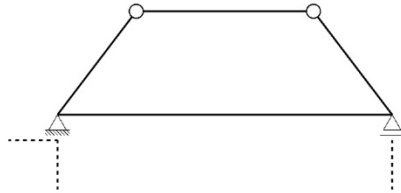
b (bars): 3
r (reactions): 4
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$3 + 4 < 2 \cdot 4$$

Unstable!

Arch-cables



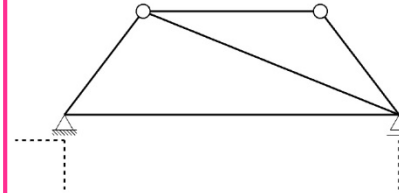
b (bars): 4
r (reactions): 3
n (nodes): 4

$$\boxed{b + r < 2 \cdot n}$$

$$4 + 3 < 2 \cdot 4$$

Unstable!

Trusses

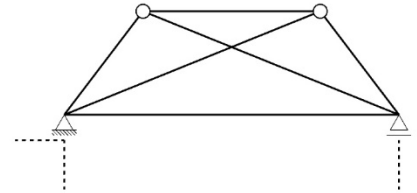


b (bars): 5
r (reactions): 3
n (nodes): 4

$$\boxed{b + r = 2 \cdot n}$$

$$5 + 3 = 2 \cdot 4$$

Stable!
 and
 statically determinate!

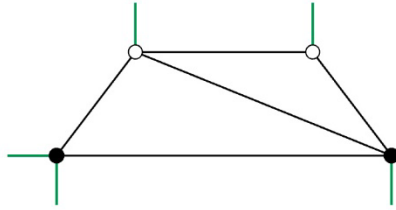


b (bars): 6
r (reactions): 3
n (nodes): 4

$$\boxed{b + r > 2 \cdot n}$$

$$6 + 3 > 2 \cdot 4$$

Stable!
 but
 statically indeterminate!



b (bars): 5

e (external forces): 5

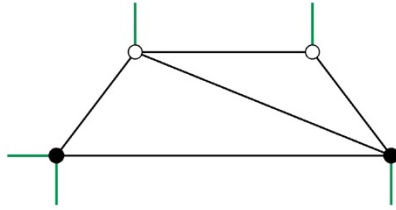
n (nodes): 4

$$\text{DOF} = b + e - (2 \cdot n)$$

$$5 + 5 - (2 \cdot 4) = 2$$

We must define the two external loads!

We have the freedom to define the two external loads!



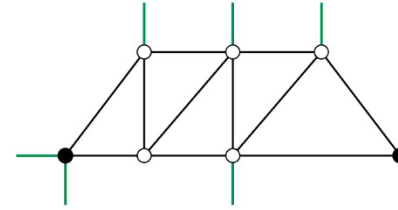
b (bars): 5
e (external forces): 5
n (nodes): 4

$$\text{DOF} = b + e - (2 \cdot n)$$

$$5 + 5 - (2 \cdot 4) = 2$$

We must define the two external loads!

We have the freedom to define the two external loads!



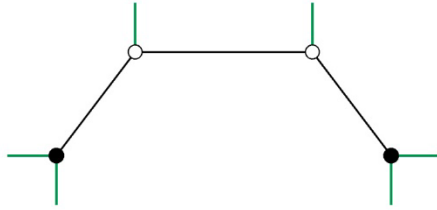
b (bars): 11
e (external forces): 7
n (nodes): 7

$$\text{DOF} = b + e - (2 \cdot n)$$

$$11 + 7 - (2 \cdot 7) = 4$$

We must define the four external loads!

We have the freedom to define the four external loads!



b (bars): 3

e (external forces): 6

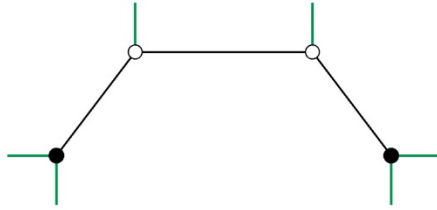
n (nodes): 4

$$\text{DOF} = b + e - (2 \cdot n)$$

$$3 + 6 - (2 \cdot 4) = 1$$

We must define one of the external loads!

We have the freedom to define one of the external loads but not the other one!



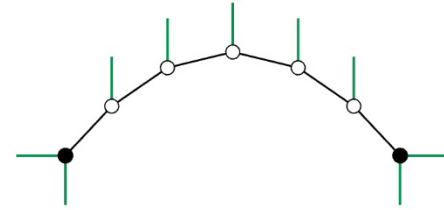
b (bars): 3
e (external forces): 6
n (nodes): 4

$$\text{DOF} = b + e - (2*n)$$

$$3 + 6 - (2*4) = 1$$

We must define one of the external loads!

We have the freedom to define one of the external loads but not the other one!



b (bars): 6
e (external forces): 9
n (nodes): 7

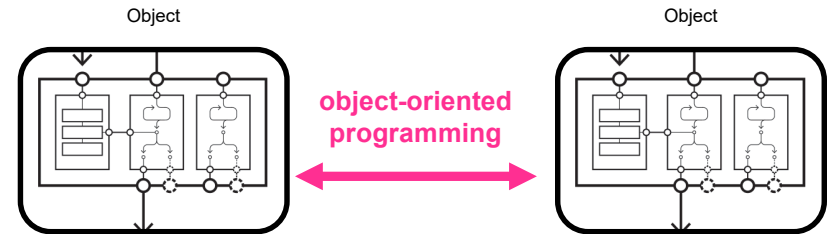
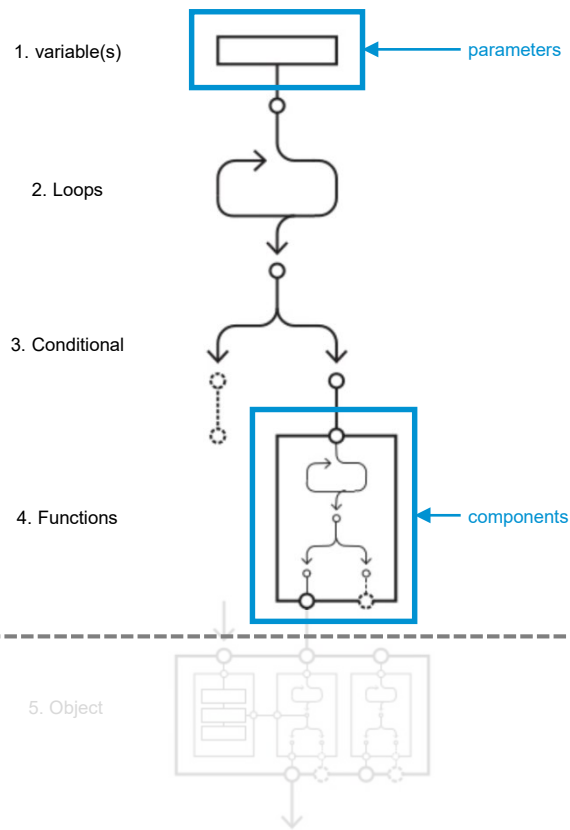
$$\text{DOF} = b + e - (2*n)$$

$$6 + 9 - (2*7) = 1$$

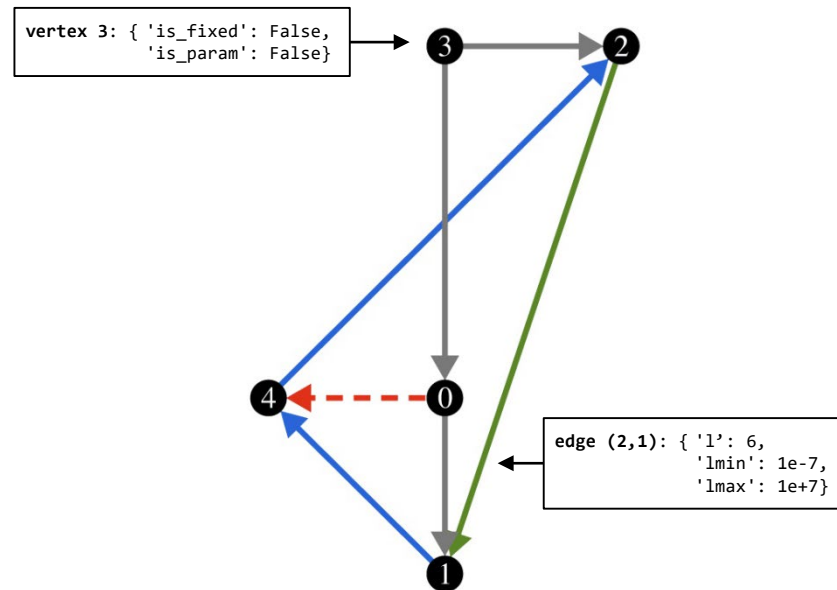
We must define one of the external loads!

We have the freedom to define only one of the external loads but not the other ones!

**linear
programming**
(Grasshopper)



force data





COMPAS

Open-source, Python-based framework for computational research and collaboration
in architecture, engineering and digital fabrication

```
60
def callback(mesh, k, callback_args):
    if not callable(callback):
        raise Exception('Callback is not callable')
    mesh.fixed = {}
    mesh.fixed[k] = True
    for key in range(mesh.nbrs):
        key_xyz = (key, mesh.vertex_coordinates(key))
        for key in mesh.vertices():
            if key in fixed:
                continue
            p = key_xyz[key]
            nbrs = mesh.vertex_neighbours(key, order)
            c = center_of_mass_polygon([key_xyz[nbr]
                                     for nbr in nbrs])
            # update mesh vertex[key]
            attr['y'] += d * (c[1] - p[1])
            attr['z'] += d * (c[2] - p[2])
        if callback:
            callback(mesh, k, callback_args)

64 def smooth_mesh_length(mesh, lmin, lmax, fixed=None):
65     """
66     """
67     if callback:
68         if not callable(callback):
69             raise Exception('Callback is not callable')
70     fixed = fixed or []
71     fixed = set(fixed)
72
```